

Pose-anchor continuity for the X3D-S wrist crop

*Smoothing and gap-handling for the wrist signal feeding the Arch 1 racket-crop sub-model.
Downstream of MMPose only, pose ground truth stays untouched at the source.*

TLDR

Use **1 Euro Filter** for jitter smoothing, with an **explicit fallback ladder** for missing or anomalous wrist samples. The smoother runs over whatever the ladder produces (real or fallback). The smoother on its own is *not enough*. Sustained dropout is a different problem from jitter and needs different machinery. Recommended ladder, in priority order: **outlier reject -> elbow-projected -> shoulder-relative -> hold-last -> null-out (after K=10 frames)**. Compute is negligible. The entire 33k-clip dataset processes in ~20 seconds. Kalman wins on pure jitter but fails on strikes and dropout. 1EF wins on consistency across regimes; details in the appendix. Two numerical divergences from the Arch 1 design doc flagged for Team: $\beta=0.3$ (not 0.5) for the centre filter based on evidence, and **outlier-rejection threshold $\tau=1.5 \times \text{torso_height}$** (physical biomechanics derivation below) instead of a looser default.

The problem and the constraint

X3D-S consumes a 13-frame, 112×112 wrist crop per stroke. Crop centre comes from the dominant-wrist keypoint (COCO 9 or 10), radius scales with torso height. MMPose currently produces this signal frame-by-frame with no temporal smoothing, no per-keypoint confidence stored, and no clamping of out-of-bounds predictions. Two failure modes show up at the consumer:

1. **Jitter**. Even when MMPose detects everything fine, the wrist coordinate jumps 3-10 px between frames at rest. Feeding this directly to the crop centre means the crop wobbles, which X3D-S then has to learn to ignore. Bad use of model capacity.
2. **Dropout / anomaly**. When the pipeline fails (< 2 people detected or != 2 on court), the entire pose for that frame is zeroed. The frame stays in the sequence. The wrist is now at coordinate (0, 0), which puts the crop in the top-left corner of the image. Even worse: anomalous-but-not-zero values (wrist predicted off-screen during a swing, or jumping 100 px between frames) sail through the same way.

Team's hard constraint: **smooth at the point of use, not at the source**. Don't interpolate inside the collation pipeline because that data feeds BST training too. Interpolated coordinates become a poison signal the model learns from. So this is a *consumer-side* component: it sits between the collated pose array and the X3D-S crop extractor, and only that.

What the 1 Euro Filter actually does

It's a low-pass filter where the cutoff frequency adapts to the rate of change of the signal. This is one equation:

$$\text{cutoff} = \min_cutoff + \beta \cdot |\text{smoothed_derivative}|$$

Then standard exponential smoothing with α derived from cutoff. That's it.

Why this shape matters for a wrist signal specifically. A badminton wrist has two regimes that fixed-bandwidth smoothers can't both serve well:

- **Rest / preparation (most of the clip)**: velocity near zero, jitter dominates. Want strong smoothing. A fixed- α EMA at $\alpha=0.3$ here looks fine.
- **Strike (~10-15 frames)**: wrist accelerates to 50+ px/frame. Want light smoothing or you lag the peak by several frames and the crop centre arrives where the wrist *was*, not where the racket *is*. That same $\alpha=0.3$ EMA now blurs the strike.

1EF resolves the tension by raising its cutoff during fast motion ($\beta \cdot |\text{derivative}|$ term kicks in) and dropping it back during quiet periods. You set *how aggressively* it does this with β . Effectively, you get the crop centre tracking faithfully through the strike and dead-still during preparation. Two filter instances per player: one each for centre_x and centre_y. Radius gets a third with different parameters (see recommendation).

Behaviour across the four dropout scenario

#	scenario	What 1EF does on its own	What's actually needed
1	Jitter (signal present, noisy)	Works as designed. β controls the strike-vs-rest tradeoff.	1EF alone, no other machinery.
2	Sustained full-pose dropout (5-10 frames)	Filter has no notion of "missing." If you feed it zeros it smooths zeros. If you feed it the last good value over and over it slowly settles there.	1EF + hold-last fallback. Filter runs on the held-last value; output stays glued to the last known position rather than diving to (0,0).
3	Wrist anomalous, rest of pose intact (OOB or large jump)	Partially harmful. The β term <i>raises</i> cutoff in response to the big derivative, so 1EF lets the anomaly <i>through</i> (it interprets it as a fast real motion).	Outlier reject + elbow-project before the filter sees the bad sample. The anomaly never reaches 1EF.
4	Full-pose dropout for 10+ frames	Same as #2 but worse. Hold-last assumption breaks down because the player has likely moved meaningfully.	1EF + hold-last for the first $\sim K=10$ frames, then null-out the X3D-S contribution for that segment. BST stream still works on whatever it has.

1EF is one component of four, not the whole answer. It owns scenario 1 cleanly and is part of the solution to 2 and 4. The fallback ladder owns scenarios 2, 3, and 4. They compose by running 1EF *over the output of the ladder*, so the filter never sees a missing sample and never sees an anomaly, only either real values or fallbacks.

The fallback ladder

Each frame, before the filter sees the wrist coordinate, the ladder runs:

Tier	Trigger	Action	Notes
1	Wrist present, frame-to-frame jump $> \tau \cdot \text{torso_height}$ ($\tau=1.5$)	Reject as anomaly, drop to tier 2	Derivation below. Above realistic peak wrist velocity ($0.6-1.0 \times \text{torso/frame}$ during pro smash), well below typical MMPose failure jumps (zeroed-frame, wrong-person swap: $3-10 \times \text{torso}$).

Tier	Trigger	Action	Notes
2	Wrist anomalous/missing, elbow valid (dominant elbow, COCO 7 or 8)	wrist = elbow + (last_good_wrist - last_good_elbow)	Preserves arm-extension geometry. Decent for 1-3 frames; degrades as the arm pose diverges.
3	Wrist + elbow both missing, shoulder valid (dominant shoulder, COCO 5 or 6)	wrist = shoulder + last_good_wrist_shoulder_offset	Last resort positional anchor. Weak but better than (0, 0).
4	Whole pose missing	Hold-last wrist coordinate	Works for short gaps where player position is stationary. Don't trust beyond ~5-10 frames.
5	Tier 4 invoked for K consecutive frames (suggest K=10)	Null-out X3D-S branch for this segment	Don't poison the fusion. Fusion module needs to handle a masked X3D-S input cleanly. Flag for Team.

The ladder stays *outside* the smoother. The smoother just sees a continuous stream of coordinates and a per-frame "real / fallback-tier-N / null" flag. The flag goes downstream as auxiliary input for X3D-S to weight or as a mask for the fusion layer.

Why these tiers in this order. Elbow-project before hold-last because during a strike the wrist moves a lot relative to the elbow but the elbow is doing something physically related. A held wrist with a moving elbow is geometrically inconsistent. Shoulder-relative below elbow because the wrist-shoulder offset varies far more than the wrist-elbow offset. Hold-last above shoulder-relative *only when the entire pose is gone* (you have no other joints to anchor against). Null-out is the cliff. There's no point in feeding X3D-S a crop centred on stale data after 10 frames; better to tell the fusion layer "ignore this branch for this segment."

Deriving $\tau=1.5 \times \text{torso_height}$ for the outlier threshold. The question is "what's the fastest the wrist can physically move between two frames?" Peer-reviewed biomechanics of the elite jump smash put peak wrist joint-centre linear velocity at **14.2 +- 1.7 m/s** across 18 experienced male players (regional, national, and international standard), measured via 18-camera Vicon motion capture at 400 Hz. Source: King, Towler, Dillon, McErlain-Naylor (2020), "A Correlational Analysis of Shuttlecock Speed Kinematic Determinants in the Badminton Jump Smash," Applied Sciences 10(4):1248, Table 2, DOI 10.3390/app10041248. The same paper's Discussion section cites Rambely et al. 2005 (ISBS conference proceedings, not directly verified here) reporting 11.7 m/s at 50 Hz capture. Lower, but likely an undersampling artefact since 50 Hz is acknowledged as inadequate for the stroke peak. The racket head reaches ~60+ m/s because of racket-length leverage, but the wrist (where our signal is) is much slower. At 30 fps (ShuttleSet broadcast frame rate), 14.2 m/s works out to 0.47 m/frame. Adult torso height (shoulder-to-hip) is ~0.45-0.55 m. So **peak wrist velocity lands at roughly 0.85-1.05× torso per frame**. $\tau=1.5$ gives ~35% margin above peak biological wrist velocity (so no real strike gets flagged), while still comfortably catching MMPose failure modes: a zeroed frame puts the wrist at (0, 0), which for a mid-frame player is ~3-8× torso away; a wrong-person swap typically jumps 2-5× torso. Both are well above 1.5.

Filter state lifecycle: reset at clip boundaries. Each stroke clip is independent, temporal continuity doesn't carry across rallies, so the smoother starts fresh each time. This is also what gives the per-clip compute numbers below their nice ceiling.

Compute

Benchmarked on a synthetic 100-frame wrist signal (Gaussian-shaped strike, $\sigma=4$ px jitter), pure Python, single core. Numbers are per-frame because that's what scales linearly with dataset size. Total assumes 33,000 clips $\times$ 100 frames $\times$ 3 signals (cx, cy, radius). 1EF compute doesn't depend on β , it's the same handful of scalar ops regardless.

Method	μ s / frame	Total dataset (sec)	Notes
EMA ($\alpha=0.3$)	0.30	3.1	Reference baseline
Savitzky-Golay (w=11, p=3)	0.91	8.5	Vectorised in SciPy; offline-only
1 Euro Filter (any β)	2.0	17.1	Recommended
Kalman (constant-velocity, filterpy)	22.7	218	Pure-Python matrix ops; 10 $\times$ the cost

The compute story isn't actually the constraint. Even Kalman at ~4 minutes total for the full dataset is fine. The X3D-S forward pass alone will dwarf this by orders of magnitude. All four methods are computationally trivial; the choice is on behaviour, not speed. Compute would only matter if we were running smoothing inside a tight per-frame inference loop in production.

Recommendation

Parameter	Value	Reasoning
Filter	1 Euro Filter, standalone PyPI package	See "Package choice" below. Uses OneEuroFilter on PyPI (maintained by original algorithm author Casiez), not MMPose's integration.
min_cutoff (centre x, y)	1.0 Hz	Standard starting value from Casiez et al. 2012; my parameter sweep showed near-flat sensitivity in the 0.5-2.0 Hz range.
β (centre x, y)	0.3 (with 0.1 as an acceptable alternative)	See β evidence below. Arch doc says 0.5, flagged for Team as a divergence.
β (radius)	0.1	Aligns with arch doc. Torso height changes slowly; aggressive tracking is wasted effort.
d_cutoff	1.0 Hz	Default; not sensitive.
Filter state	Reset per clip	Strokes are independent.
Fallback ladder	As tabulated above, K=10	See justification.

Parameter	Value	Reasoning
Outlier reject threshold τ	1.5 × torso_height per frame	Biomechanics derivation above.

β evidence. $\beta=0.5$ (matching the arch doc) (as in the arch doc) is not supported by the sweep. Segment-partitioned RMSE (whole-clip, rest, strike, worst-frame-in-strike), 20 seeded trials, mean $\pm$ std:

β	RMSE full	RMSE rest	RMSE strike	max strike err
0.05	2.99 $\pm$ 0.32	2.10 $\pm$ 0.28	4.39 $\pm$ 0.67	10.38 $\pm$ 2.19
0.10	2.97 $\pm$ 0.29	2.46 $\pm$ 0.31	3.86 $\pm$ 0.61	9.04 $\pm$ 2.12
0.20	3.17 $\pm$ 0.29	2.86 $\pm$ 0.34	3.74 $\pm$ 0.60	8.69 $\pm$ 1.65
0.30	3.31 $\pm$ 0.29	3.08 $\pm$ 0.35	3.74 $\pm$ 0.60	8.80 $\pm$ 1.56
0.50	3.49 $\pm$ 0.29	3.33 $\pm$ 0.36	3.79 $\pm$ 0.62	8.98 $\pm$ 1.39
1.00	3.70 $\pm$ 0.31	3.60 $\pm$ 0.38	3.87 $\pm$ 0.63	9.06 $\pm$ 1.28

Point estimates favour $\beta=0.2$ - 0.3 for strike-window metrics. But, and this matters, **$\beta=0.1$ and $\beta=0.3$ are not statistically separated** on the strike metrics. Welch's t-test on RMSE_strike gives $t=0.64$, $p=0.53$; on max_strike_err, $t=0.40$, $p=0.69$. The point-estimate difference is 0.125 px of RMSE, well inside the ~ 0.6 px seed-to-seed standard deviation.

Any β in $[0.1, 0.3]$ is defensible on this synthetic signal. $\beta=0.3$ is the point-estimate optimum on strike metrics; $\beta=0.1$ is marginally better on rest; $\beta=0.5$ is dominated on every metric. $\beta=0.3$ is the default, with $\beta=0.1$ as an equally acceptable alternative pending real-data validation. $\beta=0.5$ is not supported by the evidence.

Caveat on the evidence. The synthetic strike profile is a prior, not proof. Real wrist signals may have longer rest periods (shifting the optimum toward lower β), multiple strikes per clip, or asymmetric peaks. The $\beta=0.1$ vs $\beta=0.3$ tiebreak wants ~ 50 real MMPose strokes once Phase 2 collation finishes.

Package choice, correction to the arch doc. The arch doc says 1EF is "available via pip install OneEuroFilter and is also built into OpenMMLab's mmpose." Both parts need qualification:

- The PyPI package OneEuroFilter does exist and is maintained by Géry Casiez (the original 2012 algorithm author). Repository: github.com/casiez/OneEuroFilter. PyPI: pypi.org/project/OneEuroFilter. Version 0.2.1, BSD-3 license. Snyk classifies it as "inactive", fine for us, since the algorithm is mature and stable; we don't need maintenance.
- **MMPose's integration is worth not using.** In MMPose 0.x the filter lived at `mmpose.core.post_processing.temporal_filters.one_euro_filter` (plus a Smoother wrapper). GitHub issue #1176 documents a bug in that implementation: the smoothing factor is calculated wrong with respect to input fps, producing almost no visible filtering effect. The 1.x refactor (v1.0.0+, current is 1.3.2) appears to have removed the smoothing module. Issue #2726 from Sept 2023 asks where it went and has no visible resolution.
- **Recommended:** either pip install OneEuroFilter (the Casiez package) or vendor the ~ 30 -line reference implementation from Jaan Tollander's repo (github.com/jaantollander/OneEuroFilter) directly into the project. For a student-

maintained codebase I'd lean toward vendoring, one less dependency, and the code is simple enough to read end-to-end.

Implementation sketch: one wrapper class per player, holding three OneEuroFilter instances (cx, cy, radius) and a small state machine for the ladder (last-good wrist, last-good elbow offset, last-good shoulder offset, fallback-tier counter). Per-clip: instantiate, walk the frames, return smoothed (cx, cy, radius) array plus a per-frame fallback-tier flag for the fusion layer. No PyTorch needed for the smoothing itself.

Reconciliation with existing project docs

Source	States	My recommendation	Status
arch doc	1EF with min_cutoff=1.0, $\beta=0.5$ (centre), $\beta=0.1$ (radius); separate instances for cx/cy/radius	min_cutoff=1.0, $\beta=0.3$ (centre), $\beta=0.1$ (radius); separate instances	Agrees on min_cutoff, radius β, and per-signal instances. Diverges on centre β (0.3 vs 0.5), empirical; flagged for Team.
arch doc	pip install OneEuroFilter and also built into MMPose	Use standalone package (Casiez) or vendor the reference implementation	Corrects a factual inaccuracy. MMPose integration has a bug and was removed in 1.x.
arch doc	Wrist-fallback: "interpolate from elbow + previous frame's wrist direction, or carry forward the last valid crop position with EMA decay"	Full ladder: outlier reject -> elbow-project -> hold-last -> shoulder-relative -> null-out	Extends arch doc. Elbow-project aligns with existing suggestion; hold-last roughly matches EMA-carry-forward (simpler); outlier-reject, shoulder-relative, and null-out are new.
arch doc	Radius fallback (when torso keypoints occluded): torso -> shoulder width $\times$ 1.5 -> upper arm $\times$ constant -> bbox $\times$ 0.3	Not addressed. My ladder is for wrist centre, not crop radius.	Separate concern. Radius fallback is Team's design and stays as specified. This writeup doesn't overlap.

Source	States	My recommendation	Status
tuning_thoughts.md	"One Euro Filter smoothing" mentioned in the wrist crop pipeline description; no parameters specified	Same (uses 1EF)	Aligns. No divergence.
tuning_thoughts.md	Dominant-hand detection via heuristic script or player-metadata lookup	Upstream of this module; not in scope	Aligns. The wrist this smoother operates on is determined by the dominant-hand layer.

Caveats, flag these in the report

1. β validation on 50 real MMPose strokes is needed before locking the parameter. No clean reference signal exists for what the wrist 'should' be (MMPose is the source), so validation is downstream X3D-S accuracy and crop visualisations, not RMSE against a target.
2. Null-out interaction with fusion. Tier 5 (null out the X3D-S branch) requires the fusion module to handle a masked input. For late concat + MLP this is straightforward (zero out X3D-S features and route via a learned token, or fall back to BST-only prediction). For MMTM/bottleneck fusion it's less obvious — flag for you when fusion design firms up.
3. Dominant-hand handling lives upstream of this module. The wrist this modul smooths is the dominant wrist, but that determination (lefty vs righty) belongs to the metadata layer in tuning_thoughts.md — not Task 1.
4. MMPose version check. If the pipeline currently uses mmpose 0.x's built-in smoother anywhere, it may be silently doing very little filtering due to the fps bug (issue #1176). Worth a quick grep. If on 1.x, the built-in smoother is gone entirely — just add the standalone filter at the X3D-S crop step.

Kalman filter (constant velocity)

The alternative. State = $[x, vx]$, predict-then-update each frame, naturally handles missing data by skipping the update (predict-only).

The comparison is more nuanced than my first pass suggested. Sweep over a finer grid ($Q \in \{0.1, 0.5, 1, 2, 5, 10, 25, 50, 100, 250\}$, $R \in \{1, 2, 4, 9, 16, 36\}$) across three test regimes, each averaged over 20 seeds:

Test regime	Kalman best RMSE	1EF ($\beta=0.3$) RMSE
Pure jitter (constant signal + noise)	1.13 ($Q=0.1, R=36$)	3.14
Full strike signal	12.96 ($Q=250, R=1$)	3.31

Test regime	Kalman best RMSE	1EF ($\beta=0.3$) RMSE
10-frame dropout post-strike (earlier test)	46 (predict-only)	5 (hold-last + 1EF)

Kalman wins on pure jitter. A constant signal with Gaussian noise is exactly what a CV Kalman with low Q is designed for, and it beats 1EF by 3 $\times$.

Why 1EF is still the right pick, despite losing on the easy case:

- *Pure jitter is not a regime we face.* The wrist signal always contains some motion, the player moves around the court even during preparation. Kalman's "pure jitter" win is an artefact of a test, not a realistic workload.
- *On the full strike signal, Kalman fails badly.* Best RMSE=12.96 requires $Q=250$, $R=1$, which effectively says "distrust the constant-velocity model entirely, trust every measurement." At that point Kalman is roughly an unsmoothed pass-through, which defeats the purpose.
- *On dropout, Kalman's predict-only extrapolation is harmful.* Tested 10-frame dropout post-strike: Kalman extrapolates at the pre-dropout velocity ($\sim +4$ px/frame) for the entire gap, producing RMSE=46 across the dropout window vs 5 for 1EF+hold-last. The event we care about (strike) is exactly the event that violates CV's assumption.

So: Kalman handles one regime (pure jitter) well and two regimes (strike, dropout) badly. 1EF handles all three acceptably. For a signal that always contains dynamic content and occasional dropouts, consistent acceptable behaviour across regimes beats best-on-easy-case. A constant-acceleration Kalman would do better on the strike but worse on dropout; an IMM (model-switching) Kalman could handle both but at implementation cost that exceeds 1EF + ladder for marginal gain on a sub-component.

If you wanted to steel-man Kalman properly, the right comparison is IMM-Kalman with rest/strike/dropout modes vs 1EF+ladder. That's a larger research effort; not justified at this scope.

Plain EMA

Cheapest method tested (~ 0.3 μ s/frame). At $\alpha=0.5$, RMSE on the synthetic signal was 3.95, comparable to 1EF (3.0-3.5). **Where it loses:** the test signal happens to spend most of its time at moderate motion, so a moderate- α EMA isn't badly tuned for either regime. On real wrist signals with longer rest segments and sharper strikes, the EMA's fixed- α tradeoff bites harder. 1EF's whole reason for existing is precisely this regime. EMA is the right *baseline* to confirm 1EF is buying something, but not the right primary tool.

Savitzky-Golay ($w=11$, $p=3$)

Lowest RMSE on the synthetic signal (2.12) because polynomial fitting preserves peak shapes well. **Why it lost:** symmetric window means it's not causal, it uses future samples. We're doing offline preprocessing so technically this is fine, but two issues:

- Doesn't handle missing data natively; would need explicit masking.
- The 5-frame look-ahead biases the smoother toward the *symmetric* shape of the signal around each frame. Strikes in badminton are asymmetric (faster acceleration, slower deceleration), and SavGol's polynomial assumption can introduce small artefacts at the asymmetric peak.

Still a credible second choice if 1EF tuning proves fragile. Worth keeping as a fallback validation tool.

SmoothNet (Zeng et al. 2022) and learned smoothers

A small neural network trained as a refinement module on top of pose estimators. Reported gains over 1EF are real but modest (~5-10% on standard benchmarks), and the cost is a separately trained model with its own data dependency. For a downstream-only smoother on a sub-component, the engineering overhead doesn't pay back. Not recommended.

Spline interpolation

Useful only for filling gaps offline once the gap boundaries are known. Not a smoother, would have to be combined with one of the above. Cubic spline extrapolation is unstable beyond a few frames, so it doesn't help with the dropout scenarios any more than hold-last does, and adds complexity. Not recommended.